When does it pay to prepare EPR pairs in advance?

A measured study of EPR generation time
and the compiler-driven Dynamic strategy

BTP, MARS Lab, IIT Guwahati

2 November 2026

Abstract

The ACE simulator delivers an EPR pair between two neighbouring cores either *on demand* (ODG), when a request needs it, or in advance, when the compiler has planned it (*Dynamic*). The idea behind this study is that a slower pair generation hurts ODG more, so Dynamic should look better. We measured that idea directly: the generation time of a single pair on the real simulator (360 200 pairs), then 4 112 full-workload trials across 544 audited cases (shared base cases counted once) that vary the physical parameters, the time constants hidden in the simulator, and Dynamic's own options. Generation time is (4 or 5) δ/p: about 1.10 ms for one pair at the baseline. Slower generation does help Dynamic, but only up to a point, and only because of a time constant that is not physical: the 5 ms charged for every layer without inter-core requests. Without it, default Dynamic is *worse* than ODG at normal speed. We explain why with a break-even model and a funnel of what happens to every planned pair, show that the best Dynamic options validate on unseen seeds, and list what to change in the simulator and what to ask the supervisor.

Contents

1 The question and the short answers **3**

1.1 The idea . . . 3

1.2 Short answers . . . 3

1.3 How to read the numbers . . . 4

2 System and baseline **5**

2.1 How a pair is made . . . 5

2.2 The two strategies . . . 5

2.3 Workloads and routes . . . 5

2.4 What is not modelled . . . 5

3 Method **7**

3.1 Cases and seeds . . . 7

3.2 How the knobs reach the simulator . . . 7

3.3 Pre-registration and deviations . . . 7

4 Anatomy of a delivery **9**

4.1 One request, message by message . . . 9

4.2 The formula . . . 9

4.3 What a hit costs . . . 10

5 The knobs of generation time **11**

6 Does slower generation help Dynamic? **13**

6.1 The efficiency sweep . . . 13

6.2 Two limits on the hit rate (H4) . . . 13

6.3 The classical-delay sweep . . . 14

7 The hidden constant: the empty-layer gap **15**

7.1 What the gap is . . . 15

7.2 The result . . . 15

7.3 Why: what happens to each planned pair . . . 16

7.4 A break-even model 16
7.5 Can we predict readiness? 19

8 Time constants and chip-scale speeds **20**
8.1 Do only ratios matter? 20
8.2 The retry test 20
8.3 Chip-scale regime 21
8.4 Distance 22

9 Tuning Dynamic **23**
9.1 One option at a time 23
9.2 The lead depends on the speed and the gap 23
9.3 Do the chosen options hold on unseen seeds? 25

10 Routes, memory, coherence and other workloads **27**
10.1 Routes 27
10.2 Memory per core 28
10.3 Coherence 28
10.4 Other workloads 30

11 Hypothesis scorecard **31**

12 How to think about it **32**

13 Recommendations **33**

14 Limitations and threats to validity **34**
14.1 The single-link settings that were hard to simulate 34

A Reproducing the study **35**

B File map **36**

C Provenance **37**

Chapter 1

The question and the short answers

1.1 The idea

Creating an EPR pair between two cores is probabilistic: it can fail and must be retried, and each try needs several classical messages. ODG pays this time *inside* the request. Dynamic uses the circuit's known future requests to start the pair earlier, in a quiet moment, so that the request finds it ready. The idea we tested is simple: *the longer a pair takes to make, the more ODG suffers and the more Dynamic gains.*

1.2 Short answers

1. **What is the generation time?** For one hop, $T_{\text{gen}} = (4 \text{ or } 5)\,\delta\,/\,p$, where δ is the classical message delay (100 μs by default), $p = 0.407$ is the success chance of one attempt, and the factor is 4 for a hop whose source core has the lower number and 5 for the reverse hop. The mean is 1.10 ms (0.972 ms forward, 1.220 ms reverse) after 2.44 attempts on average. This was measured on 360 200 pairs and matched the formula in every probe for the number of attempts (Chapter 4).

2. **Which knobs change it?** The classical delay, and anything that changes p: memory efficiency, detector efficiency, the BSM success rate, and link loss. Memory excitation rate matters only below about 5 kHz (the default is 80 MHz). Coherence time, fidelity settings, gate settings and the empty-layer gap do not change it (Chapter 5).

3. **Does slower generation help Dynamic?** Yes at first. On QFT with $M = 4$ the latency gain over ODG rises from 46.8% at the baseline to a peak of 59.4% when ODG generation takes 2.8 ms, then falls to 51.2% at 10.6 ms, because pairs no longer finish before they are needed. The *absolute* saving keeps growing, from 0.62 ms to 5.85 ms per hop (Chapter 6).

4. **What is the catch?** The gain depends on a 5 ms constant that gives every empty layer five milliseconds. With it, preparations finish in the quiet time (9.9 ms between launch and request, against 1.11 ms to make a pair). With no gap, only 19.7% of requests find a ready pair, memory contention pushes the others 0.59 ms later, and Dynamic is −19.5% against ODG on QFT (Chapter 7).

5. **Can Dynamic be improved?** Yes, and the improvement holds on unseen seeds. Letting Dynamic use all memories (C=M) cuts QFT latency by 16–24%, and borrowing pairs or demand-first cuts QAOA latency by 7–14%. Many popular options hurt, and a longer lead hurts badly (Chapter 9).

6. **What should we be careful about?** Absolute latencies rest on two assumed constants

($\delta = 100\,\mu$s, taken from a metro-scale paper, and the 5 ms gap). Every claim here is a ratio, a regime, or a mechanism, and the report says which (Chapter 14).

1.3 How to read the numbers

- **Latency** is the mean time per hop from when a request becomes eligible until its EPR pair is delivered. Data teleportation and local gates are not simulated.
- **Gain** is $100 \times (1 - \text{Dynamic/ODG})$ on the same route, memories, seeds and settings. A negative gain means Dynamic is worse.
- **Hit** means the pair was ready before the request was first submitted.
- **Makespan** is the time to finish the whole workload. **Communication time** is the makespan minus the empty-layer gaps.
- Intervals are 95% Student-t intervals over seeds. Seeds change the simulator's random numbers, not the circuit or its mapping.

Chapter 2

System and baseline

2.1 How a pair is made

A hop moves a qubit between two neighbouring cores on a 4×4 mesh. The pair is made between one memory in each core, with a Bell-state measurement station (BSM) halfway between them. One *attempt* runs four steps: a pairing handshake, a NEGOTIATE message that agrees when both cores emit, both cores emit a photon towards the BSM, and a heralding message back from the BSM that says whether it worked. The photons themselves fly in about 125 ps. Every other step is a classical message that takes δ. An attempt succeeds with probability

$$p \;=\; r_{\mathrm{BSM}} \left(\eta_{\mathrm{mem}}\, \eta_{\mathrm{det}}\, \eta_{\mathrm{fibre}}\right)^2, \qquad r_{\mathrm{BSM}} = 0.5,\ \eta_{\mathrm{mem}} = \eta_{\mathrm{det}} = 0.95,\ \eta_{\mathrm{fibre}} \approx 1,$$

so $p = 0.407$ at the baseline. The expression comes from the engine's BSM code and is checked in Chapter 5.

2.2 The two strategies

ODG starts the pair when the request is admitted. **Dynamic** runs an offline planner over the known sequence of requests. For each request it chooses an earlier layer in which to start the pair, within limits on memories per core and on preparations started per layer. At run time a controller launches each preparation when its layer opens, and the request later adopts the ready pair in place ("handoff").

2.3 Workloads and routes

Mappings are frozen, so a route is the only difference between two traces of the same workload. The main workloads are QFT-96 and QAOA-96; Draper-96, CDKM-48 and a random circuit test generality. **Routes.** XY is the base. `history` is the project's chosen route: it routes the longest moves first over shortest paths, picks the path that adds least overload, and adds a small penalty for cores used in earlier layers. The 2026-10-30 traces labelled "greedy" were checked to be pure greedy (all moves reproduced), not `history`.

2.4 What is not modelled

Data teleportation, local gate time, and circuit fidelity are not simulated. A fidelity threshold of 0.01 never rejects a pair, so a pair that would be useless on hardware still counts as delivered.

Setting	Baseline	Meaning
Memories per core, M	3 or 4	communication memories for EPR pairs
Compiler memories, C	$M-1$	most prepared pairs a core may hold; one memory stays free for ODG-style requests
Planner lead	2 layers	the planner first tries to start a pair two layers before its request
Lookahead	8 layers	how far back it may look if that layer is full
Generation capacity	C	preparations that may start per core per layer
Compiler reservation	50 ms	how long a prepared pair may hold its memory
Admission	ready-first	requests with a ready pair go first
Consumption	handoff	the request takes over the prepared pair in place
Classical delay δ	100 μs	per message; set by a formula in `router_net_topo_adaptive.py`
Empty-layer gap	5 ms	time charged for a layer with no inter-core request
Request retry, compiler retry	1 ms, 1 ms	wait before a refused request or preparation is tried again
Coherence time	1 s	lifetime of a stored pair

Table 2.1: Baseline, identical to the 2026-10-30 routing study except $C = M - 1$.

Workload	Qubits	Layers	With traffic	Empty	Hops
QFT	96	766	229	537	3833
QAOA	96	733	328	405	7913
Draper adder	96	950	291	659	2661
CDKM adder	48	715	169	546	402
Random circuit	48	3341	2217	1124	10336

Table 2.2: Workloads. “Empty” layers contain only local gates and no inter-core request.

Chapter 3

Method

3.1 Cases and seeds

A *case* is one setting, for example QFT, `history`, $M = 4$, $C = 3$, memory efficiency 0.6. For each seed the whole workload is simulated twice, once with ODG and once with Dynamic. Each of those is a *trial*. All 4 112 unique trials completed every request and passed the simulator's physical audit, which checks ownership and ordering of every memory. Seeds 60–69 (or 60–65, or 60–63 for the screening experiments) are the primary seeds. Options were chosen only on seeds 60–63, and then tested on seeds 100–105 that tuning never saw.

3.2 How the knobs reach the simulator

No file under `ace/` was changed. A knob is either a value in a copy of the JSON configuration (efficiencies, distance, coherence, ...) or a constant replaced in memory before the simulation is built: the classical delay (the base of the 100 μs formula), the empty-layer gap, and the two retry waits. A test checks that default knobs reproduce the original configuration. The new runner reproduced earlier manual runs bit for bit (QFT, $M = 4$, $C = 3$: ODG 1.3249 ms, Dynamic 0.7013 ms), and those had reproduced the published 2026-10-30 numbers exactly at $C = 4$.

3.3 Pre-registration and deviations

Ten hypotheses, each with a rule that would falsify it, were committed before any sweep ran (`plan.md`). Three things changed afterwards, and all three are logged in the plan with the date:

1. The matrix was reduced from 912 to 562 cases because the machine (8 GB, swapping) ran about ten times slower than planned. Seeds and grids were cut; every hypothesis stayed covered.
2. Three single-link settings that did not finish in 15 minutes (dark counts 10^6 and 10^8 /s, coherence 0.1 ms) were stopped and rerun with a time limit; one finished (Section 14.1).
3. Two worker pools overlapped on a few cases and produced three spurious failure markers (one pool removed a configuration file another still needed). All three flagged cases were checked: each has a complete result set, with 0 audit violations and 100% completion. The pools were then separated, and the final audit accepts all 544 cases.

Two experiments were added after seeing results, and are labelled as such: the distance test (a request from you) and the retry test, which was designed to test an inference made from the

chip-scale results (Chapter 8).

Chapter 4

Anatomy of a delivery

4.1 One request, message by message

Figure 4.1 shows measured events for one hop. In (a) the request needs a new pair. The guard is the reservation handshake, a REQUEST and an APPROVE. The first attempt then fails, so the pairing handshake starts again, and the second attempt succeeds. In (b) the pair already exists, because it was made about 10 ms earlier by a preparation. The request still pays the guard, then a pairing step, and the pair is handed over when the `INFORM_EP` message arrives.

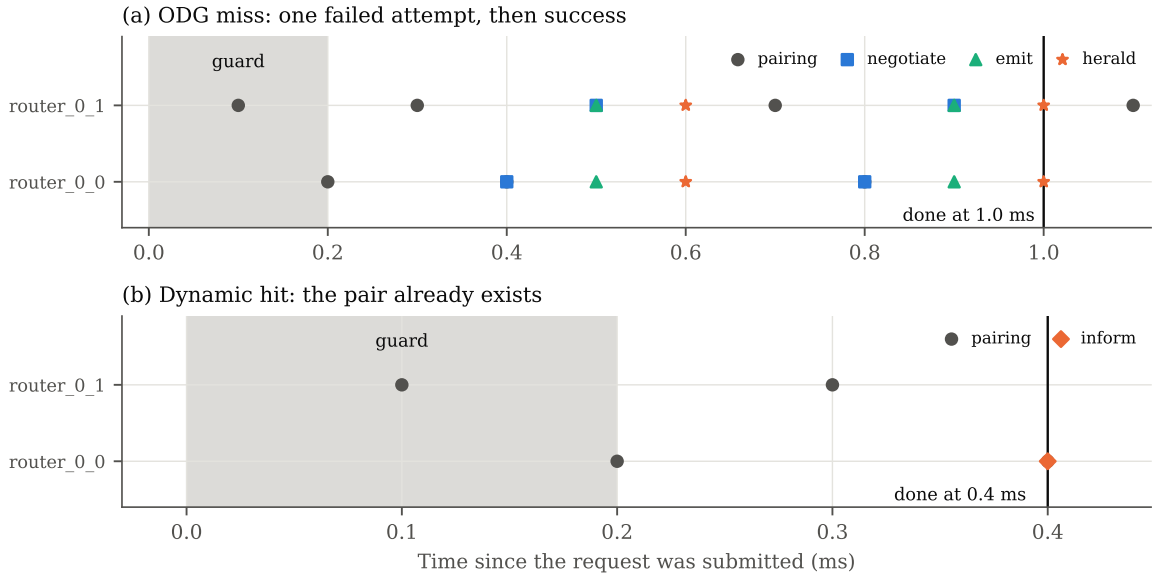


Figure 4.1: Measured events of one request on $0 \to 1$ (core 0 to core 1) at $\delta = 100\,\mu$s, with time zero at submission. Markers: pairing messages, NEGOTIATE, photon emission, heralds and `INFORM_EP`. Source: `anatomy.csv`.

4.2 The formula

Each failed attempt repeats the pairing handshake, so an attempt costs the same every time: exactly 4δ (0.4 ms) on a forward hop and exactly 5δ (0.5 ms) on a reverse hop, in every one of the 108 probes (minimum and maximum per-attempt time were equal). The reason is which node starts first. The extra δ appears when the node that sends NEGOTIATE (the one with the

greater name) starts its protocol one message later than its partner. With k attempts geometric with parameter p,

$$T_{\text{gen}} = k\,c\,\delta, \quad c \in \{4, 5\}, \quad \mathbb{E}[T_{\text{gen}}] = \frac{c\,\delta}{p}.$$

The measured mean is 0.972 ms forward and 1.220 ms reverse, against 0.98 and 1.23 predicted. The mean number of attempts is 2.44 measured and 2.46 predicted. Figure 4.2 compares the whole distribution.

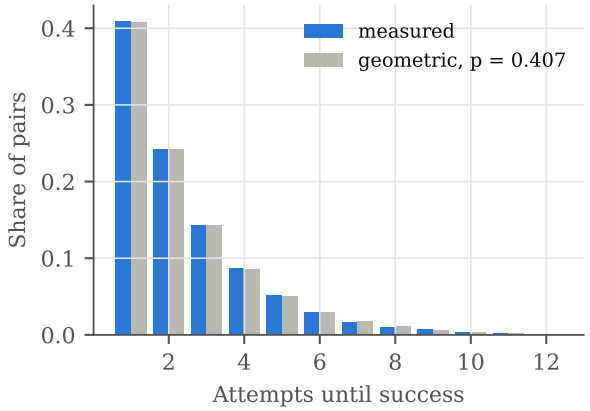


Figure 4.2: Attempts until success on a forward hop, 54,000 pairs, against the geometric law.

4.3 What a hit costs

A hit is not free. It costs the guard (2δ) plus one or two further δ: 0.4 ms on a forward hop and 0.5 ms on a reverse hop, with no physical attempt, whatever p is. At the baseline the mean hit hop takes 0.45 ms, against 1.32 ms for ODG.

Finding. Everything in a delivery is classical messaging except the photon flight. The generation time is therefore a statement about δ and p, and $\delta = 100\,\mu$s is an assumption taken from a metro-scale paper, not a chip-scale value.

Chapter 5

The knobs of generation time

We changed one physical knob at a time and measured single-hop generation time on both hop directions: 108 probes and 360 200 pairs in total. **What we can say.**

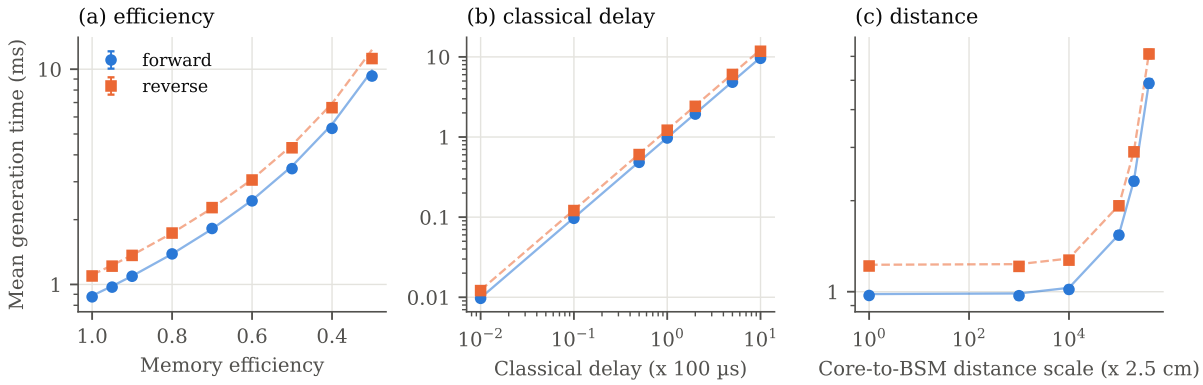


Figure 5.1: Mean generation time against efficiency, classical delay and distance. Points: measured, with 95% intervals; lines: the formula. Blue: forward hop; orange: reverse hop.

Knob changed	Values tried	Pairs	Within tolerance	Worst error (%)
fibre attenuation	0 to 200	30000	8/10	−8.9
none (baseline)	–	54000	6/6	−1.2
BSM success rate	0.1 to 0.4	24000	8/8	−4.6
classical delay scale	0.01 to 10	36000	12/12	−4.2
coherence time	0.0002 to 0.1	30000	10/10	−1.6
dark counts	1000 to 1e+06	6200	4/4	+13.6
detector efficiency	0.4 to 1	30000	10/10	+4.1
link length scale	1000 to 400000	30000	10/10	−1.7
empty-layer gap	0 to 20	12000	4/4	−1.6
gate fidelity	0.9	6000	2/2	−1.6
measurement fidelity	0.9	6000	2/2	−1.6
memory efficiency	0.3 to 1	48000	14/16	−9.0
memory excitation rate	1000 to 1e+06	36000	8/12	+99.3
raw pair fidelity	0.85 to 0.99	12000	4/4	−1.6

Table 5.1: Every knob tried. “Within tolerance”: measured mean within the interval plus 2% of the formula’s prediction.

- The number of attempts matches $1/p$ in all 108 probes, for efficiency, detector efficiency, BSM rate and fibre loss alike.
- The time matches in 100 of 108. Four misses are efficiency 0.3 and attenuation 200 dB/m, where about 1% of requests exceed the 50 ms request window and restart; the clock then sees only the final, shorter try. That is a measurement effect, not physics. Four misses are memory excitation at 1 and 2 kHz, where the memory itself limits each attempt (up to +99%). This is a true exception to our second hypothesis. The default of 80 MHz is far from it.
- Distance does nothing at chip or module scale: the time is flat out to 25 m, 5% higher at 250 m, and rises sharply only at kilometre scale, because both the delay and the fibre loss then grow.
- Dark counts of 10^6 /s agree with the formula; 10^8 /s and a coherence time of 0.1 ms were too slow to simulate (Section 14.1).

The equivalence of causes (H3). Efficiency of the memory, efficiency of the detectors and the BSM success rate can each give the same p. We checked that the *workload* result is the same whichever one is changed: 8 of 36 comparisons exceed two standard errors (1.8 expected by chance), none survives a Bonferroni correction, and the largest difference is 2.4%. Chapter 6 therefore varies only memory efficiency.

Chapter 6

Does slower generation help Dynamic?

6.1 The efficiency sweep

Memory efficiency from 1.0 to 0.3 moves ODG generation time from 1 to 10.6 ms. All other settings stay at the baseline, with the 5 ms gap. **Reading the figure.** The latency gain rises,

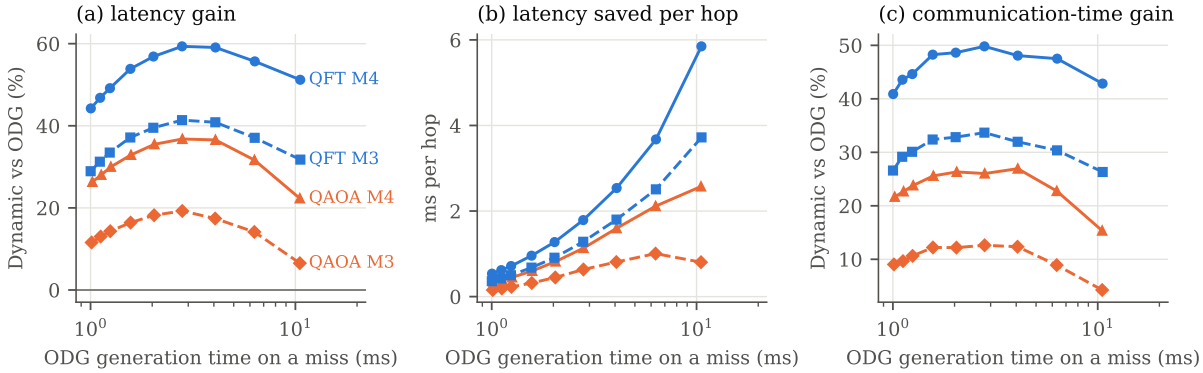


Figure 6.1: Dynamic against ODG on the `history` route as generation slows. (a) Relative latency gain; (b) absolute saving per hop; (c) gain in communication time. M is memories per core; lines are labelled at the right.

peaks at efficiency 0.6 in every context, and then falls: 46.8% → 59.4% → 51.2% on QFT $M = 4$, and 28.0% → 36.8% → 22.3% on QAOA $M = 4$. Panel (b) shows why the fall is not a failure of the idea: the saving per hop grows from 0.62 ms to 5.85 ms on QFT. The gain falls in relative terms because ODG's latency grows faster than Dynamic's once pairs stop being ready in time. The measure that falls is the *share of accepted preparations that are ready*, from 100% at the baseline to between 53% and 68% at efficiency 0.3.

6.2 Two limits on the hit rate (H4)

- **The planner's memory limit** fixes how many pairs can be prepared at all. The blocked share is the same at every efficiency (spread 0.000 in all four contexts), and at fast generation it equals the miss rate.
- **Preparation time against slack** decides whether a prepared pair is ready. It starts to bite only when generation takes a sizeable part of the time between launch and request.

Case	Eff.	ODG gen. (ms)	ODG (ms)	Dyn. (ms)	Gain (%)	Saved (ms)	Comm. gain (%)	Hit (%)
QAOA-96 M3	1	1.01	1.335	1.180	11.6	0.15	9.0	38.1
QAOA-96 M3	0.95	1.12	1.454	1.265	13.0	0.19	9.7	38.1
QAOA-96 M3	0.8	1.57	1.941	1.622	16.4	0.32	12.2	38.0
QAOA-96 M3	0.6	2.80	3.296	2.661	19.3	0.63	12.6	36.9
QAOA-96 M3	0.4	6.31	7.110	6.104	14.1	1.01	8.9	28.6
QAOA-96 M3	0.3	10.51	12.297	11.492	6.5	0.80	4.3	20.4
QAOA-96 M4	1	1.02	1.254	0.924	26.3	0.33	21.7	53.7
QAOA-96 M4	0.95	1.13	1.364	0.981	28.0	0.38	22.7	53.7
QAOA-96 M4	0.8	1.58	1.821	1.222	32.9	0.60	25.6	53.4
QAOA-96 M4	0.6	2.81	3.083	1.949	36.8	1.13	26.0	51.4
QAOA-96 M4	0.4	6.31	6.679	4.565	31.7	2.11	22.8	40.9
QAOA-96 M4	0.3	10.51	11.533	8.960	22.3	2.57	15.3	28.5
QFT-96 M3	1	1.00	1.245	0.885	28.9	0.36	26.6	55.2
QFT-96 M3	0.95	1.11	1.357	0.933	31.3	0.42	29.2	55.2
QFT-96 M3	0.8	1.56	1.825	1.147	37.1	0.68	32.4	55.0
QFT-96 M3	0.6	2.80	3.099	1.817	41.4	1.28	33.7	52.5
QFT-96 M3	0.4	6.33	6.764	4.257	37.1	2.51	30.4	43.7
QFT-96 M3	0.3	10.57	11.716	7.995	31.8	3.72	26.3	37.6
QFT-96 M4	1	1.00	1.211	0.675	44.2	0.54	40.9	73.6
QFT-96 M4	0.95	1.11	1.322	0.703	46.8	0.62	43.6	73.6
QFT-96 M4	0.8	1.56	1.776	0.819	53.9	0.96	48.3	73.4
QFT-96 M4	0.6	2.80	3.013	1.225	59.4	1.79	49.8	70.2
QFT-96 M4	0.4	6.34	6.599	2.924	55.7	3.67	47.5	57.3
QFT-96 M4	0.3	10.56	11.422	5.572	51.2	5.85	42.8	47.9

Table 6.1: Efficiency sweep, `history`, gap 5 ms. Gen. is ODG generation time on a miss.

Chapter 7 turns the second limit into a prediction.

6.3 The classical-delay sweep

Scaling the delay scales every time in a delivery equally. For QFT the relative gain is flat from

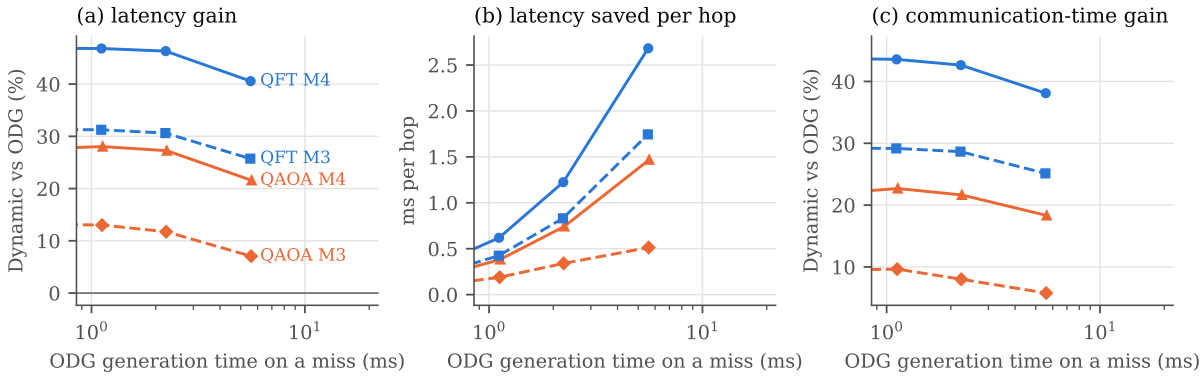


Figure 6.2: The same plot for the classical delay, ×0.1 to ×5.

×0.1 to ×2 (46.8% → 46.3%) and falls only at ×5 (40.6%): the whole system scales together, and the hit rate is identical. For QAOA the gain at ×0.1 is lower (20%, against 28% at the baseline), for the reason given in Chapter 8. The absolute saving is proportional to δ: 0.06 ms per hop at $\delta = 10\,\mu$s and 2.7 ms at 500 μs.

Finding. Slower generation helps Dynamic in absolute terms throughout, and in relative terms up to about 3 ms of generation time. The best case is where ODG generation takes 2.8 ms.

Chapter 7

The hidden constant: the empty-layer gap

7.1 What the gap is

The compiler cuts a circuit into layers. Some layers need no inter-core request, only local gates. The simulator does not model gate time. It charges every such layer exactly 5 ms (`EMPTY_LAYER_GAP_MS`), and zero for a layer that has requests. The value has been in the code since the first import and the code comment calls it "preserved from the legacy manager". We found no physical reason for it. It is large: it accounts for 2.7 s of the 3.3 s QFT makespan. It also supplies idle time in which Dynamic can prepare pairs.

7.2 The result

Case	Eff.	gap 0	0.5 ms	1 ms	2 ms	5 ms	20 ms
QAOA-96 M3	0.4	−5 / 18	−4 / 20	–	–	+14 / 29	–
QAOA-96 M3	0.6	−13 / 21	−7 / 23	–	–	+19 / 37	–
QAOA-96 M3	0.95	−26 / 19	−12 / 25	–	–	+13 / 38	–
QAOA-96 M4	0.4	+11 / 27	+14 / 29	+18 / 31	+22 / 34	+32 / 41	+32 / 41
QAOA-96 M4	0.6	+2 / 29	+8 / 32	+15 / 36	+24 / 42	+37 / 51	+38 / 53
QAOA-96 M4	0.95	−13 / 27	−1 / 33	+14 / 43	+26 / 52	+28 / 54	+28 / 54
QFT-96 M3	0.4	−0 / 20	+4 / 24	–	–	+37 / 44	–
QFT-96 M3	0.6	−16 / 18	−6 / 23	–	–	+42 / 52	–
QFT-96 M3	0.95	−31 / 16	−8 / 27	–	–	+31 / 55	–
QFT-96 M4	0.4	+15 / 23	+20 / 28	+26 / 32	+36 / 40	+56 / 57	+66 / 71
QFT-96 M4	0.6	−0 / 22	+10 / 28	+23 / 38	+40 / 51	+59 / 70	+61 / 73
QFT-96 M4	0.95	−20 / 20	+4 / 33	+27 / 53	+44 / 70	+47 / 74	+47 / 74

Table 7.1: Gain in % / hit rate in %, for each gap, `history`.

At the baseline efficiency and no gap, default Dynamic is worse than ODG: −19.5% on QFT. At low efficiency and $M = 4$ it still wins without the gap (QFT efficiency 0.4: +15%), because ODG is then so slow that even a 20% hit rate pays; at $M = 3$ it only breaks even (QFT efficiency 0.4: −0.5%). From 5 ms to 20 ms the gain does not change, so 5 ms already supplies enough time.

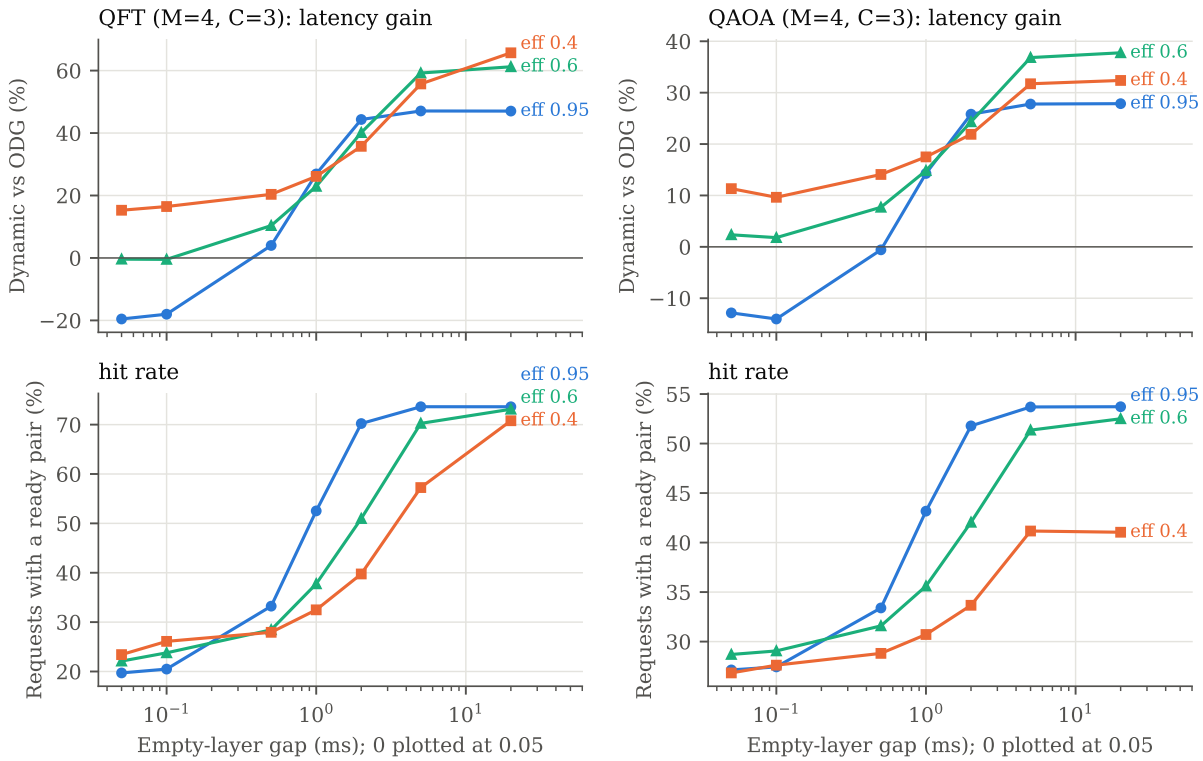


Figure 7.1: Dynamic's latency gain (top) and hit rate (bottom) against the gap, $M = 4$, $C = 3$, for three memory efficiencies. Zero is plotted at 0.05 ms.

7.3 Why: what happens to each planned pair

Read the table from the top. The planner promises the same 73.6 preparations per 100 requests at every gap, because it counts layers and does not know how long a layer lasts. With no gap:

1. Slack falls from 9.9 ms to 1.4 ms, while making a pair still takes 1.11 ms plus the guard.
2. A pair that is still being made when its request starts is not used, and the request generates its own.
3. Preparations and live requests now compete for the same four memories on each core. The protocol rejects preparations (80 rejection events per 100 requests, against 0), so only 35.9% are accepted and 6.3% of pairs are made and then expire unused.
4. Requests that miss wait 0.59 ms for a free memory, against 0.10 ms with the gap and 0.008 ms for ODG, which never competes with preparations.

ODG's latency does not change with the gap (1.325 ms in all three columns), so the extra delay comes from what Dynamic adds. The mechanism that preparations take memories from live requests is inferred from this evidence and from the rejection counts. We did not trace individual memory conflicts.

7.4 A break-even model

Let h be the hit rate, L_{hit} the hop latency of a hit, L_{rest} the mean latency of every other request, and L_{ODG} the ODG latency. Dynamic's latency is $hL_{\text{hit}} + (1 - h)L_{\text{rest}}$, so Dynamic beats ODG

Per 100 requests	gap 5 ms	gap 1 ms	gap 0
Preparations planned	73.6	73.6	73.6
Launched	73.6	72.2	64.6
Accepted by the protocol	73.6	60.1	35.9
Pair actually made	73.6	60.1	35.9
Used by a request	73.6	57.2	29.5
Ready before the request (hit)	73.6	52.5	19.7
Made but expired unused	0.0	2.8	6.3
Rejections (events)	0.0	44.9	80.4
Times (ms)			
Time from launch to request (slack)	9.9	2.4	1.4
Time to make a pair	1.11	1.10	1.11
Hit hop latency	0.448	0.459	0.579
Non-hit hop latency	1.409	1.600	1.904
Wait for a free memory, non-hit	0.100	0.309	0.592
ODG hop latency	1.325	1.325	1.325
ODG wait for a free memory	0.008	0.008	0.008
Dynamic hop latency	0.701	0.968	1.584

Table 7.2: QFT, $M = 4$, $C = 3$, efficiency 0.95, four seeds: what happens to the planned preparations, and what a request pays, at three gaps.

exactly when

$$h \;>\; h^{*} \;=\; \frac{L_{\text{rest}} - L_{\text{ODG}}}{L_{\text{rest}} - L_{\text{hit}}}.$$

This is algebra, so it fits all 188 cases by construction. What it adds is the size of the two quantities. L_{hit} is almost constant, about 0.45 ms. What changes is the extra cost of a non-hit over ODG, $L_{\text{rest}} - L_{\text{ODG}}$: about 0.08 ms with the gap, and about 0.5 ms without it. So the break-even hit rate rises from 8.7% to 40.4%, while the measured hit rate falls from 73.6% to 19.7%.

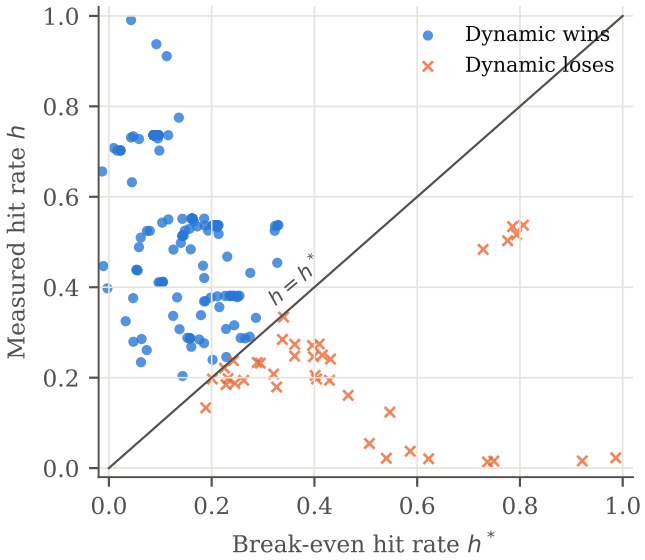


Figure 7.2: Measured hit rate against break-even hit rate in all 188 cases of the `history` route. Every loss lies below the diagonal.

Workload	Gap (ms)	Hit (%)	Hit cost	Non-hit cost	ODG	Break-even (%)	Dynamic
QAOA-96	0	27.1	0.515	1.918	1.362	39.6	1.537
QAOA-96	0.1	27.4	0.526	1.944	1.363	40.9	1.554
QAOA-96	0.5	33.4	0.491	1.811	1.362	34.0	1.370
QAOA-96	1	43.2	0.462	1.702	1.361	27.5	1.167
QAOA-96	2	51.8	0.454	1.609	1.362	21.4	1.010
QAOA-96	5	53.7	0.452	1.600	1.362	20.7	0.984
QAOA-96	20	53.7	0.452	1.599	1.363	20.6	0.983
QFT-96	0	19.7	0.579	1.830	1.325	40.4	1.584
QFT-96	0.1	20.5	0.598	1.813	1.325	40.1	1.564
QFT-96	0.5	33.2	0.506	1.653	1.325	28.6	1.272
QFT-96	1	52.5	0.459	1.532	1.325	19.3	0.968
QFT-96	2	70.2	0.448	1.420	1.325	9.8	0.737
QFT-96	5	73.6	0.448	1.409	1.325	8.7	0.701
QFT-96	20	73.6	0.448	1.410	1.325	8.9	0.702

Table 7.3: Break-even model at efficiency 0.95, $M = 4$. Costs in ms.

In the 40 cases where Dynamic loses, the median hit rate is 20% and the median break-even is 41%.

7.5 Can we predict readiness?

For an accepted preparation, the pair is ready if its attempts finish inside the slack s between its launch window and its request. With k geometric, $P(\text{ready} \mid s) = 1 - (1-p)^{\lfloor s/c\delta \rfloor}$. We read the slack distribution from each run (its 10th, 50th and 90th percentile and its maximum), so the model's only inputs are p, δ and the hop direction. The model is accurate for QFT (mean

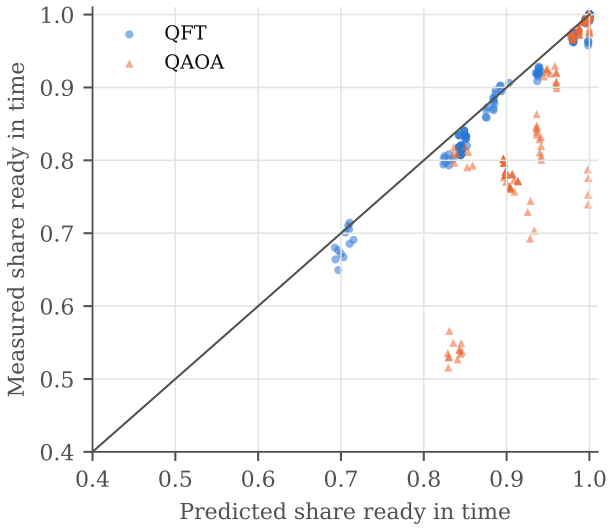


Figure 7.3: Predicted against measured share of accepted preparations that were ready, in 656 trials with a gap of at least 5 ms, lead 2 and 1 s coherence.

absolute error 0.7 points) and over-predicts QAOA at the slowest settings (mean error 3.7 points, up to 30 points at efficiency 0.3). Our guess is that the percentiles we kept cannot represent QAOA's short-slack tail. We did not test that. The model is one reason we trust the "two limits" picture, but it is not a complete explanation for QAOA.

Chapter 8

Time constants and chip-scale speeds

8.1 Do only ratios matter?

If every time constant scaled together, the latency would scale and the gain would not change. The scale test multiplies the delay, the gap, and the window sizes by 0.01, 0.1 and 2.

- **QFT**: the hit rate is identical at every scale (73.62%) and the gain is 47.1% throughout; ODG latency per unit scale is constant to better than 1%.
- **QAOA**: the gain moves from +28% to −59% at ×0.01. About 1% of QAOA requests need a reservation retry under Dynamic (ODG has none), and each retry costs at least the fixed 1 ms retry wait. At ×0.01 the mean latency is 0.014 ms, so one 1 ms retry per 100 requests is as large as the whole latency.

8.2 The retry test

This explanation was an inference when we first made it, so we tested it: we scaled the two retry waits together with the delay. QAOA at ×0.01: −59% with the fixed 1 ms waits, and +24%

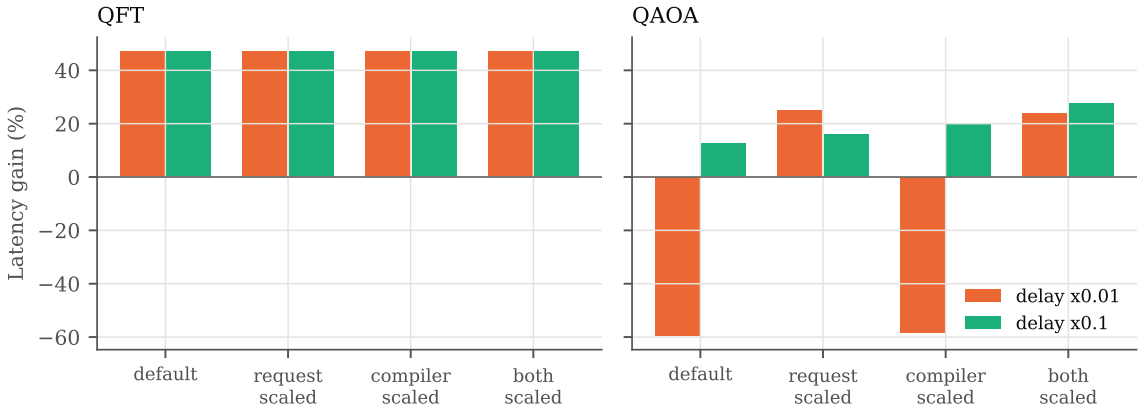


Figure 8.1: Latency gain at delay ×0.01 and ×0.1 when the request retry, the compiler retry, or both are scaled with the delay.

when both are scaled. The slowest hop falls from 1.77 ms to 0.11 ms. QFT has no retries and does not change.

Workload	Delay ×	Request retry (ms)	Compiler retry (ms)	ODG (ms)	Dyn. (ms)	Gain (%)	Slowest hop (ms)
QAOA-96	0.01	1	1	0.0137	0.0218	−59.4	1.769
QAOA-96	0.01	1	0.01	0.0137	0.0216	−58.3	1.525
QAOA-96	0.01	0.01	1	0.0137	0.0102	25.0	0.102
QAOA-96	0.01	0.01	0.01	0.0137	0.0104	23.8	0.112
QAOA-96	0.1	1	1	0.1362	0.1193	12.5	1.523
QAOA-96	0.1	1	0.1	0.1362	0.1088	20.2	1.515
QAOA-96	0.1	0.1	1	0.1362	0.1145	16.0	1.163
QAOA-96	0.1	0.1	0.1	0.1362	0.0989	27.4	0.980
QFT-96	0.01	1	1	0.0133	0.0070	47.1	0.070
QFT-96	0.01	1	0.01	0.0133	0.0070	47.1	0.070
QFT-96	0.01	0.01	1	0.0133	0.0070	47.1	0.070
QFT-96	0.01	0.01	0.01	0.0133	0.0070	47.1	0.070
QFT-96	0.1	1	1	0.1325	0.0701	47.1	0.700
QFT-96	0.1	1	0.1	0.1325	0.0701	47.1	0.700
QFT-96	0.1	0.1	1	0.1325	0.0701	47.1	0.700
QFT-96	0.1	0.1	0.1	0.1325	0.0701	47.1	0.700

Table 8.1: Retry test, $M = 4$, $C = 3$, efficiency 0.95. Slowest hop shows the retry stalls.

Finding. At fast classical control, a 1 ms retry wait destroys Dynamic on QAOA. It is a fixed constant of the simulator, not a property of the strategy. Any chip-scale study of Dynamic must scale it.

8.3 Chip-scale regime

Workload	Delay ×	Gap (ms)	ODG (ms)	Dyn. (ms)	Latency gain (%)	Makespan gain (%)	Hit (%)
QAOA-96	0.01	0	0.0137	0.0328	−140.1	−87.1	1.1
QAOA-96	0.01	0.1	0.0137	0.0225	−64.9	−21.2	51.6
QAOA-96	0.01	1	0.0137	0.0218	−59.5	−2.8	53.4
QAOA-96	0.01	5	0.0137	0.0228	−67.2	−0.7	53.7
QAOA-96	0.1	0	0.1362	0.2141	−57.2	−36.0	12.4
QAOA-96	0.1	0.1	0.1362	0.1613	−18.4	−9.9	25.0
QAOA-96	0.1	1	0.1363	0.1084	20.5	5.4	53.5
QAOA-96	0.1	5	0.1362	0.1088	20.2	1.5	53.7
QFT-96	0.01	0	0.0133	0.0225	−69.5	−46.1	2.3
QFT-96	0.01	0.1	0.0133	0.0070	47.1	7.7	73.6
QFT-96	0.01	1	0.0133	0.0070	47.1	0.9	73.6
QFT-96	0.01	5	0.0133	0.0070	47.1	0.2	73.6
QFT-96	0.1	0	0.1325	0.1988	−50.0	−27.8	5.4
QFT-96	0.1	0.1	0.1325	0.1395	−5.3	2.8	28.5
QFT-96	0.1	1	0.1325	0.0701	47.1	7.7	73.6
QFT-96	0.1	5	0.1325	0.0701	47.1	1.8	73.6

Table 8.2: Fast control with short gaps, `history`, $M = 4$, $C = 3$.

With the delay at 1 μs, QFT keeps its 47% gain as long as the gap is at least 0.1 ms, but the saving per hop is 0.006 ms and the makespan gain is under 8%: almost all of the time is the gap. QAOA stays negative at these speeds with the default retry waits (see above). The size of Dynamic's benefit therefore depends jointly on δ, the gate time that a realistic gap would encode, and the retry wait.

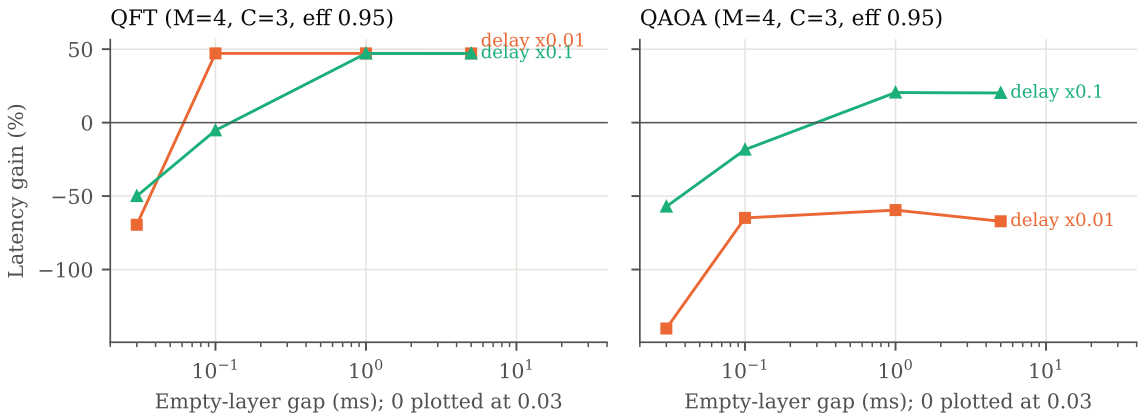


Figure 8.2: Latency gain against the gap at delay ×0.01 (1 µs) and ×0.1 (10 µs).

8.4 Distance

You asked what happens with real core-to-core distances. The configured distance between cores is 5 cm (0.025 m from each core to the BSM halfway). From 5 cm to 30 m, nothing changes beyond

Workload	Core-to-core distance	ODG (ms)	Dyn. (ms)	Gain (%)	Hit (%)
QAOA-96	1m	1.3612	0.9831	27.8	53.7
QAOA-96	6.6m, delay base 1 µs	0.0141	0.0232	−64.5	53.7
QAOA-96	6.6m	1.3625	0.9825	27.9	53.7
QAOA-96	30m	1.3660	0.9847	27.9	53.7
QFT-96	1m	1.3235	0.7012	47.0	73.6
QFT-96	6.6m, delay base 1 µs	0.0137	0.0073	46.9	73.6
QFT-96	6.6m	1.3209	0.7022	46.8	73.6
QFT-96	30m	1.3272	0.7039	47.0	73.6

Table 8.3: Real link lengths at $M = 4$, $C = 3$, efficiency 0.95, four seeds.

seed noise: light crosses 30 m in 150 ns, which is 670 times smaller than the 100 µs added to every message. Distance would matter only at kilometre scale.

Chapter 9

Tuning Dynamic

9.1 One option at a time

We changed one Dynamic option at a time, in 8 contexts (QFT and QAOA, $M = 3$ and 4, efficiency 0.95 and 0.4), on seeds 60–63. **What it shows.**

Case	Eff.	Default (ms)	Best option	Change (%)	Worst option	Change (%)	No effect	Better	Worse
QAOA-96 M3	0.4	6.070	borrow	−10.5	lead=3	+115	8	2	15
QAOA-96 M3	0.95	1.269	demand-first	−7.6	lead=6	+102	8	3	15
QAOA-96 M4	0.4	4.537	borrow	−13.9	lead=3	+140	7	2	16
QAOA-96 M4	0.95	0.984	borrow	−7.0	lead=8	+109	7	3	18
QFT-96 M3	0.4	4.215	C=3	−21.1	lead=4	+159	8	3	15
QFT-96 M3	0.95	0.933	C=3	−16.3	lead=8	+159	13	1	13
QFT-96 M4	0.4	2.902	C=4	−23.2	lead=4	+218	8	2	16
QFT-96 M4	0.95	0.701	C=4	−16.0	lead=8	+174	12	1	15

Table 9.1: Best and worst option in each context, and how many options changed latency by more than 2%.

- **The default is close to a local optimum.** Of the options tried, only two families help anywhere.
- **C=M helps QFT** by 16–23%: letting Dynamic use the memory that was reserved for ODG-style requests. It does not help QAOA, where it is worse by 2–8%.
- **Borrowing pairs and demand-first help QAOA** by 5–14%. They change nothing for QFT.
- **Lookahead and generation capacity (8) do nothing**, which agrees with the group's earlier finding. FIFO admission is also identical.
- **A longer lead hurts, often badly:** lead 4 made QFT 8–218% slower and QAOA 60–74% slower. Other options that hurt: swap, runtime planning, first-hop-only preparation, and longer reservations (+0 to +138%).

9.2 The lead depends on the speed and the gap

The best lead is 1, 2, 3 or 4, depending on the case: QFT at slow generation prefers lead 3 at gap 5 ms (2.49 ms against 2.90 ms for lead 2), and lead 4 with no gap (1.01 ms against 1.58 ms at baseline efficiency). The jump at lead 4 matches QFT's layout, which has about one active layer every four layers: the lead is counted in layers, but what matters is the time available. A planner

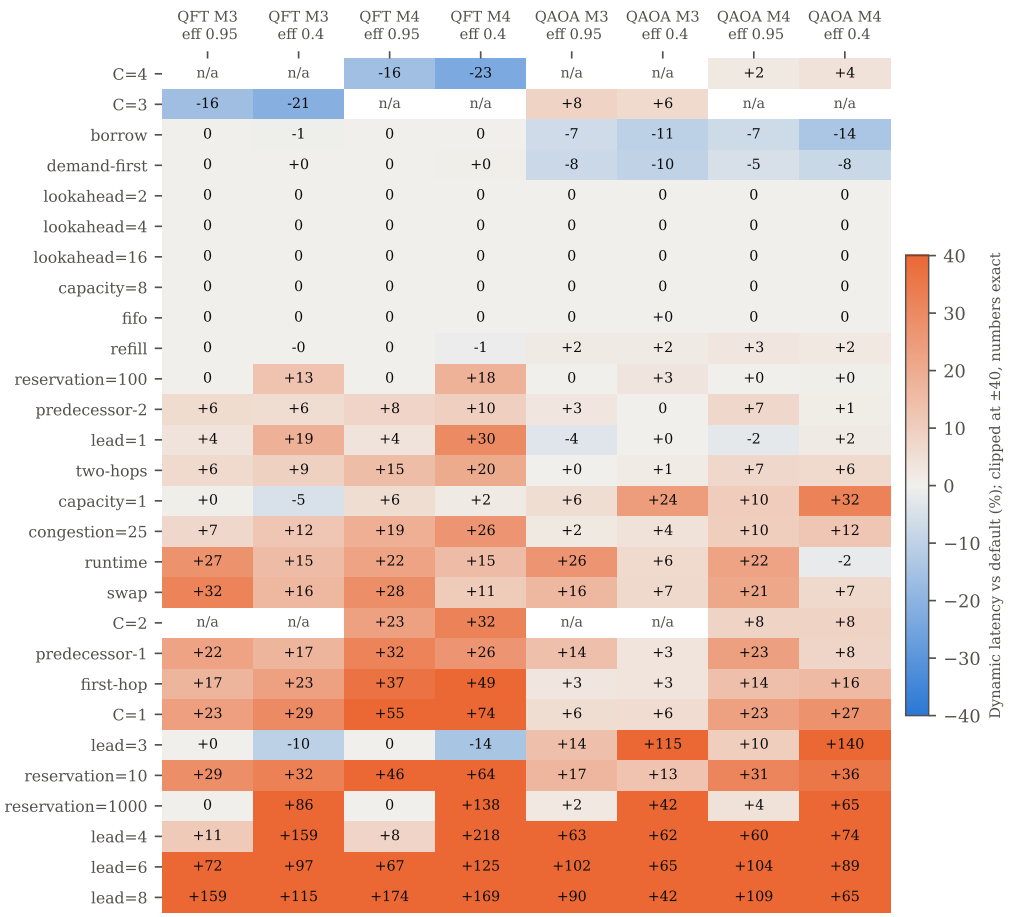


Figure 9.1: Change in Dynamic's latency against the default, in %. Blue: better; orange: worse. The scale is clipped at ±40%; the printed numbers are exact. "n/a": that option is the default in that context.

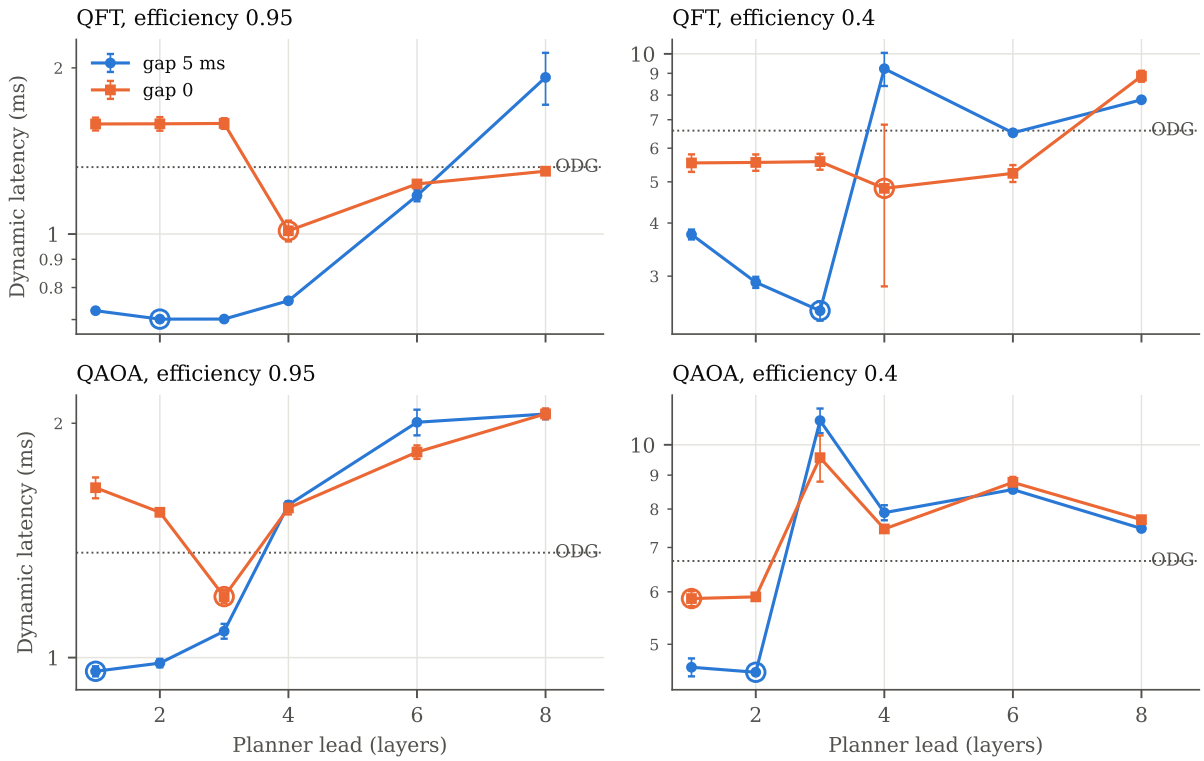


Figure 9.2: Dynamic latency against the planner lead ($M = 4$). Rings mark the best lead; the dotted line is ODG.

that counts milliseconds, or active layers, would remove this tuning. Beyond lead 4, memory fills with waiting pairs and latency rises.

9.3 Do the chosen options hold on unseen seeds?

We picked, in each context, the option with the lowest mean latency if it beat the default by more than the two 95% intervals added together, and ran it on seeds 100–105. The rule was written before the validation seeds were run. All 8 of 8 options held, with effects within about one point

Case	Eff.	Option	Tuning (%)	Unseen seeds (%)	Default vs ODG	Chosen vs ODG	Holds
QAOA-96 M3	0.4	borrow	−10.5	−10.7 ± 1.0	−14.0	−23.2	yes
QAOA-96 M3	0.95	demand-first	−7.6	−6.9 ± 1.3	−13.3	−19.3	yes
QAOA-96 M4	0.4	borrow	−13.9	−14.2 ± 1.2	−31.8	−41.5	yes
QAOA-96 M4	0.95	borrow	−7.0	−6.9 ± 1.6	−28.4	−33.4	yes
QFT-96 M3	0.4	C=3	−21.1	−22.4 ± 2.3	−36.8	−51.0	yes
QFT-96 M3	0.95	C=3	−16.3	−16.1 ± 0.5	−31.3	−42.4	yes
QFT-96 M4	0.4	C=4	−23.2	−24.2 ± 2.8	−55.7	−66.4	yes
QFT-96 M4	0.95	C=4	−16.0	−15.9 ± 1.0	−46.9	−55.3	yes

Table 9.2: Validation. “vs ODG” is the change in Dynamic’s latency against ODG on the same seeds.

of tuning, and every chosen option still beats ODG. Two cautions: all seeds share one mapping per workload, and C=M removes the memory that was reserved for ODG-style requests, which matters if demand traffic can be heavier than in these workloads.

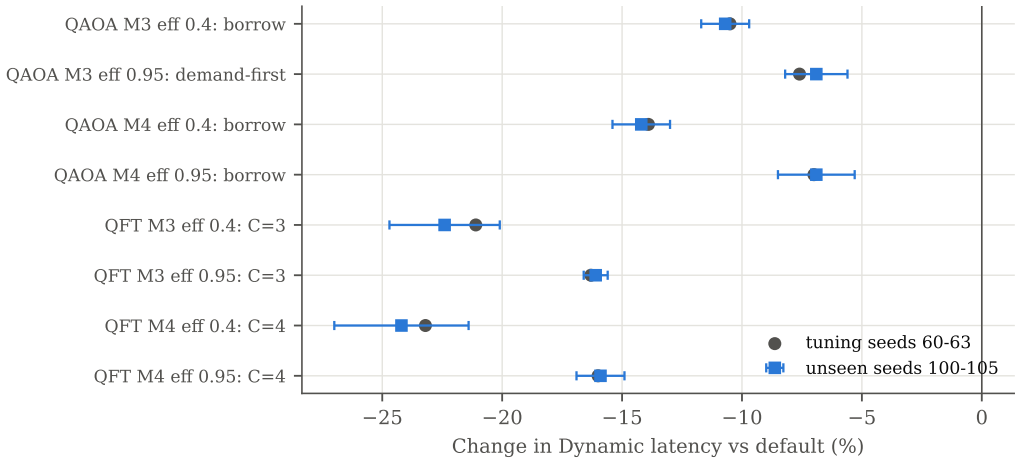


Figure 9.3: Latency change from the default: tuning seeds (grey) and unseen seeds (blue, with 95% interval).

Chapter 10

Routes, memory, coherence and other workloads

10.1 Routes

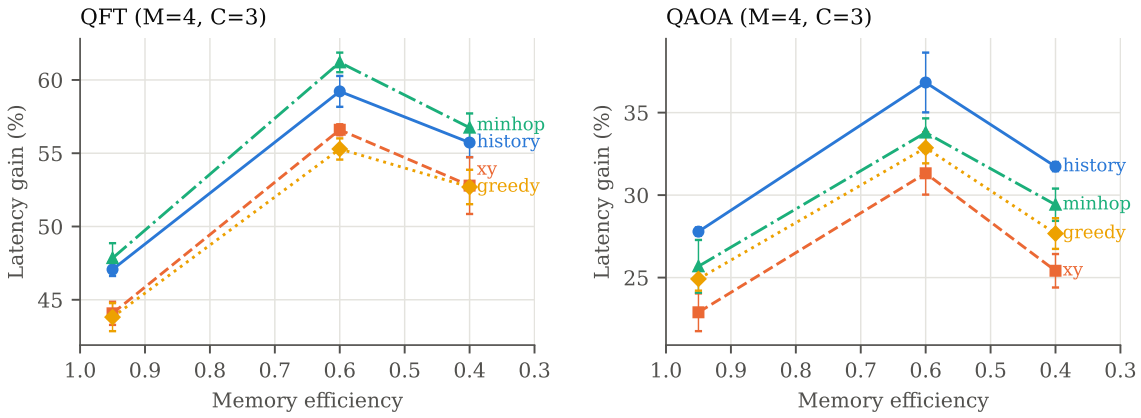


Figure 10.1: Latency gain by route against memory efficiency ($M = 4$, four seeds, 95% intervals).

Workload	Eff.	XY	greedy	history	minhop
QAOA-96	0.95	22.9 ± 1.1	24.9 ± 0.7	27.8 ± 0.3	25.7 ± 1.6
QAOA-96	0.6	31.3 ± 1.3	32.9 ± 0.9	36.8 ± 1.8	33.8 ± 0.9
QAOA-96	0.4	25.4 ± 1.0	27.7 ± 0.9	31.7 ± 0.3	29.4 ± 1.0
QFT-96	0.95	44.1 ± 0.8	43.8 ± 1.0	47.1 ± 0.4	47.8 ± 1.0
QFT-96	0.6	56.6 ± 0.4	55.3 ± 0.7	59.2 ± 1.1	61.2 ± 0.7
QFT-96	0.4	52.8 ± 1.9	52.7 ± 1.2	55.7 ± 1.0	56.7 ± 1.0

Table 10.1: Latency gain in % by route, $M = 4$.

`history` beats XY at every efficiency in all four contexts by 2 to 6 points (36 of 36 points of the efficiency sweep). It is best on QAOA. On QFT, `minhop` is slightly better (e.g. 47.8% against 47.1% at baseline, 61.2% against 59.2% at efficiency 0.6), and the differences exceed the interval. Its advantage over XY grows with slower generation only for QAOA $M = 4$ (Spearman $\rho = 0.82$, $p = 0.007$), so our tenth hypothesis is only partly supported.

10.2 Memory per core

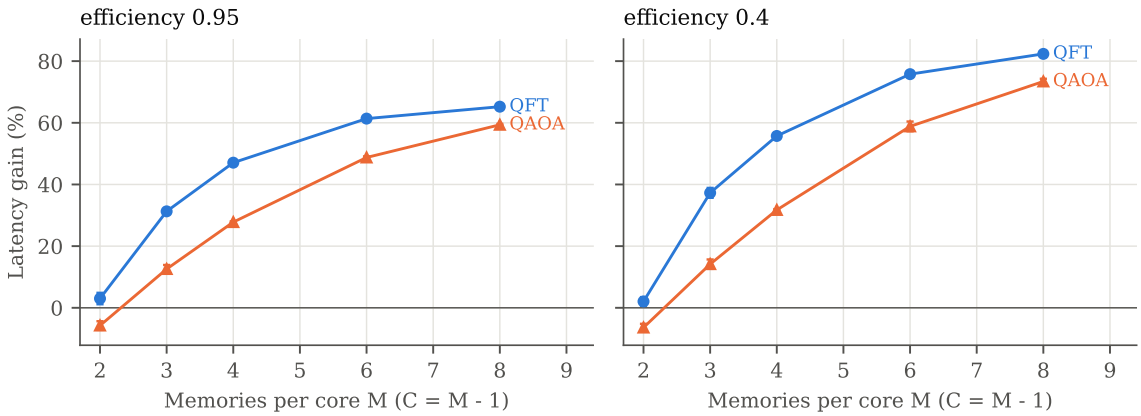


Figure 10.2: Latency gain against M (with $C = M - 1$).

Workload	Eff.	M	ODG (ms)	Dyn. (ms)	Gain (%)	Hit (%)	Planner-blocked (%)
QAOA-96	0.95	2	1.80	1.91	−5.8	18.7	81.3
QAOA-96	0.95	3	1.45	1.27	12.6	38.1	61.9
QAOA-96	0.95	4	1.36	0.98	27.8	53.7	46.3
QAOA-96	0.95	6	1.33	0.68	48.7	77.5	22.4
QAOA-96	0.95	8	1.33	0.54	59.4	91.1	8.8
QAOA-96	0.4	2	8.87	9.44	−6.4	13.3	81.3
QAOA-96	0.4	3	7.07	6.07	14.2	28.7	61.9
QAOA-96	0.4	4	6.65	4.54	31.7	41.2	46.3
QAOA-96	0.4	6	6.51	2.68	58.8	60.4	22.4
QAOA-96	0.4	8	6.50	1.73	73.4	71.9	8.8
QFT-96	0.95	2	1.56	1.52	3.0	28.8	71.2
QFT-96	0.95	3	1.36	0.93	31.3	55.2	44.8
QFT-96	0.95	4	1.32	0.70	47.1	73.6	26.4
QFT-96	0.95	6	1.32	0.51	61.4	93.8	6.2
QFT-96	0.95	8	1.32	0.46	65.2	99.1	0.9
QFT-96	0.4	2	7.74	7.58	2.1	24.6	71.2
QFT-96	0.4	3	6.72	4.22	37.3	43.8	44.8
QFT-96	0.4	4	6.55	2.90	55.7	57.3	26.4
QFT-96	0.4	6	6.52	1.58	75.8	72.9	6.2
QFT-96	0.4	8	6.52	1.15	82.3	78.5	0.9

Table 10.2: Memory per core. “Planner-blocked” is the share of requests the planner could not prepare.

Memory per core is the strongest lever we found. On QFT at the baseline efficiency, the gain rises from 3% at $M = 2$ to 65% at $M = 8$, because the planner-blocked share falls from 71% to 1%. At $M = 2$, Dynamic is worse than ODG for QAOA.

10.3 Coherence

Pairs wait about 9 ms in memory at baseline (a two-layer lead spans two 5 ms gaps). At coherence 100 ms only fidelity changes (0.94 → 0.84–0.87). At 10 ms pairs start to expire: the QFT gain falls from 47% to 20% and QAOA turns negative. At 1 ms Dynamic is worse than ODG by 55–99%.

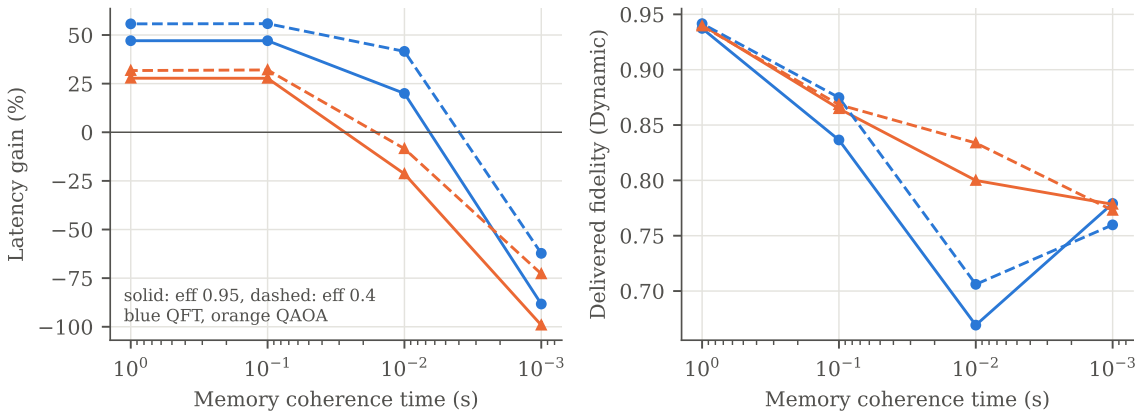


Figure 10.3: Dynamic's latency gain and delivered fidelity against the memory coherence time.

Workload	Eff.	Coherence (s)	Gain (%)	ODG fid.	Dyn. fid.	Expired pairs	Hit (%)
QAOA-96	0.95	1	27.8	0.950	0.940	0	53.7
QAOA-96	0.95	0.1	27.7	0.948	0.865	0	53.7
QAOA-96	0.95	0.01	−21.4	0.932	0.800	1079	24.1
QAOA-96	0.95	0.001	−99.2	0.786	0.778	1908	1.6
QAOA-96	0.6	1	36.8	0.950	0.939	63	51.4
QAOA-96	0.6	0.1	36.9	0.948	0.859	57	51.5
QAOA-96	0.6	0.01	−21.0	0.932	0.799	1702	24.6
QAOA-96	0.6	0.001	−92.6	0.786	0.777	2994	1.6
QAOA-96	0.4	1	31.7	0.950	0.939	602	41.2
QAOA-96	0.4	0.1	32.0	0.948	0.868	601	41.1
QAOA-96	0.4	0.01	−8.5	0.932	0.834	2293	19.4
QAOA-96	0.4	0.001	−72.8	0.786	0.773	3822	2.1
QFT-96	0.95	1	47.1	0.950	0.937	0	73.6
QFT-96	0.95	0.1	47.1	0.948	0.837	0	73.6
QFT-96	0.95	0.01	20.0	0.932	0.669	557	46.8
QFT-96	0.95	0.001	−88.3	0.786	0.779	1401	1.4
QFT-96	0.6	1	59.2	0.950	0.939	44	70.3
QFT-96	0.6	0.1	59.2	0.948	0.853	44	70.2
QFT-96	0.6	0.01	35.0	0.932	0.656	500	49.8
QFT-96	0.6	0.001	−55.1	0.786	0.771	1710	2.2
QFT-96	0.4	1	55.7	0.950	0.942	243	57.3
QFT-96	0.4	0.1	55.8	0.948	0.875	230	57.5
QFT-96	0.4	0.01	41.6	0.932	0.706	663	44.7
QFT-96	0.4	0.001	−62.2	0.786	0.760	2365	3.7

Table 10.3: Coherence time, $M = 4$, $C = 3$.

The fidelity of Dynamic is not monotone at 1 ms (0.76–0.78) because almost no prepared pair survives, and most deliveries are then on demand. ODG's own fidelity also falls at 1 ms.

10.4 Other workloads

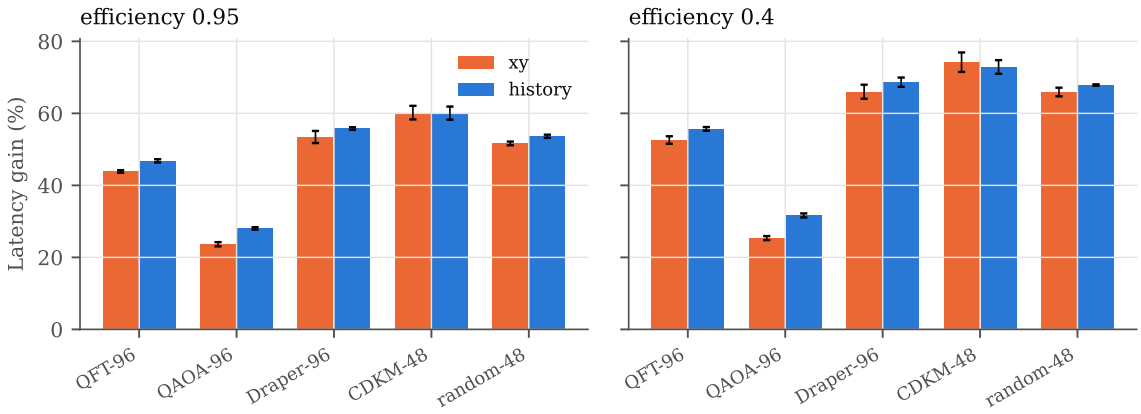


Figure 10.4: Latency gain for five workloads ($M = 4$, $C = 3$).

Workload	Route	Eff.	ODG (ms)	Dyn. (ms)	Gain (%)	Hit (%)	Blocked (%)
CDKM-48	history	0.95	1.31	0.52	60.1 ± 1.8	91.2	8.7
CDKM-48	xy	0.95	1.30	0.52	60.2 ± 1.9	92.4	7.2
CDKM-48	history	0.4	6.43	1.74	72.9 ± 1.9	66.1	8.7
CDKM-48	xy	0.4	6.56	1.69	74.2 ± 2.7	68.9	7.2
Draper-96	history	0.95	1.31	0.58	55.8 ± 0.4	85.1	14.9
Draper-96	xy	0.95	1.30	0.61	53.4 ± 1.7	81.4	18.6
Draper-96	history	0.4	6.47	2.03	68.7 ± 1.3	67.0	14.9
Draper-96	xy	0.4	6.50	2.21	66.0 ± 1.9	63.9	18.6
random-48	history	0.95	1.31	0.61	53.6 ± 0.4	81.0	12.3
random-48	xy	0.95	1.31	0.63	51.7 ± 0.5	78.9	14.4
random-48	history	0.4	6.46	2.07	67.9 ± 0.2	65.2	12.3
random-48	xy	0.4	6.49	2.21	65.9 ± 1.2	63.9	14.4

Table 10.4: The three additional workloads, four seeds.

Dynamic wins on every workload we tried, and by more on the three additional workloads (54–60% at the baseline). `history` is ahead of XY on Draper and the random circuit; on CDKM the two differ by less than their intervals.

Chapter 11

Hypothesis scorecard

ID	Hypothesis	Verdict	Evidence
H1	Generation time is (4 or 5) δ/p.	**Mostly supported**	Attempts: 110/110 probes inside tolerance. Time: 102/110; the 8 misses are 4 at memory excitation 1–2 kHz (H2) and 4 where requests longer than the 50 ms window restart (efficiency 0.3, attenuation 200 dB/m).
H2	Only δ and the factors of p matter.	**Partly falsified**	Coherence, fidelity, gap and dark counts $\leq$ 1000/s change nothing (all within 14.7%). Memory excitation below 5 kHz does matter (+99% at 1 kHz).
H3	Equal p gives equal results whatever its cause.	**Supported in practice**	8 of 36 comparisons exceed two standard errors (1.8 expected by chance); none survives a Bonferroni correction; the largest difference is 2.4%.
H4	Two separate limits on the hit rate.	**Supported**	The planner-blocked share is constant in efficiency (spread 0.000 in all four cases) and equals the miss rate at fast generation. The ready-in-time share of accepted pairs falls only below efficiency 0.6.
H5	Absolute saving grows; relative gain peaks.	**Supported**	All four contexts peak at an interior efficiency (0.6); the absolute saving grows with generation time in 3 of 4 (QAOA M3 turns at 0.4).
H6	The gap decides how much Dynamic gains.	**Supported, strongly**	With no gap, default Dynamic loses to ODG at normal speed (−20% on QFT M4/C3). The hit rate responds to the gap in every context. Beyond 5 ms the gain stops changing.
H7	Only ratios of time constants matter.	**Falsified for QAOA, then explained**	QFT: hit rate identical at every scale. QAOA at delay ×0.01: −59%, caused by the fixed 1 ms retry waits; scaling them restores +24% (retry test).
H8	The best lead depends on generation time.	**Supported**	Best leads found: 1, 2, 3, 4. They depend on both the gap and the generation speed, and leads above 4 are never best.
H9	Short coherence removes the gain.	**Supported**	Coherence 100 ms changes only fidelity; 10 ms cuts the QFT gain from 47% to 20% (hits fall as pairs expire); 1 ms makes Dynamic 55–99% worse than ODG.
H10	History's advantage grows with slower generation.	**Partly supported**	History beats XY at every efficiency in all 4 contexts, by 2–6 points. The advantage grows significantly only for QAOA M4 ($\rho = 0.82$, $p = 0.007$); it is flat for QFT.

Table 11.1: The ten hypotheses of `plan.md`, each judged by the rule written before the sweeps.

Chapter 12

How to think about it

1. **Dynamic is ordering ahead.** A request that finds its pair ready pays only the reservation handshake and a handover: about 0.4–0.5 ms here, whatever generation costs. ODG pays that plus generation, about 1.1 ms at the baseline.
2. **A pair is useful only if it is ready in time, and only if preparing it did not delay someone else.** The first needs slack longer than generation. The second needs spare memory.
3. **Slow generation helps until it eats the slack.** Savings per hop keep growing, but past about 3 ms of generation time the share of pairs that are ready in time falls, and the relative gain with it.
4. **The gap is the slack.** The 5 ms charged per empty layer is where Dynamic gets its time. Take it away and Dynamic needs a lead counted in time, not in layers.
5. **Memory is the real limit.** At baseline speed, almost every planned pair is ready; the pairs that are missing are the ones the planner could not fit into the memories. More memory per core gains more than any tuning.
6. **Fixed constants hide in simulators.** 100 μs per message, 5 ms per empty layer, and 1 ms per retry together decide the absolute numbers, and two of them decided a sign in this study.

Chapter 13

Recommendations

1. **State results as ratios and regimes.** The absolute latencies depend on constants that are not hardware values. Quote the gain, the hit rate and the break-even, and say which constants were used.
2. **Ask the supervisor for the target platform**, then fix the three constants together: the classical delay, a gap built from gate times, and the retry waits. Our results give the sensitivity for each.
3. **For the default configuration**, keep lead 2 (lead 3 for slow generation), and use C=M for QFT-like circuits and borrowing or demand-first for QAOA-like circuits. All three held on unseen seeds.
4. **Change the planner to count time.** The best lead differs across contexts because the planner counts layers. A slack in milliseconds would remove the tuning and the collapse at no gap. This is the most promising code change we see.
5. **Add more memory per core before tuning anything else.** Moving from $M = 4$ to $M = 6$ is worth more than every option we tried.
6. **Next experiments:** a planner that counts time; gate times for the gap; more than one mapping per workload; a fidelity threshold; torus topology; and data teleportation timing.

Chapter 14

Limitations and threats to validity

- **One mapping per workload.** Seeds vary the simulator's randomness only. Intervals therefore understate the uncertainty about other circuits or mappings. Five workloads help, but they are five mappings.
- **Assumed constants.** The delay, the gap and the retry waits are not calibrated to hardware.
- **Reduced matrix.** Seeds are 4 to 10 per setting, and some grids are coarse. Gains are quoted with their intervals.
- **Inferences, not tests.** The statement that preparations take memory from live requests comes from the funnel and the unchanged ODG latency, not from tracing conflicts. The reason the readiness model over-predicts QAOA was not tested.
- **Single-link gaps.** Dark counts of 10^8 /s and coherence 0.1 ms could not be simulated in a practical time (Section 14.1).
- **Fidelity.** No pair is rejected for low fidelity, so a Dynamic pair that waits 9 ms still counts. The reported delivered fidelity is a mean over pairs, not a circuit success probability.

14.1 The single-link settings that were hard to simulate

Dark counts of 10^6 and 10^8 per second and a coherence time of 0.1 ms did not finish 3000 samples in the first screen (15 minutes each), because they create events every microsecond or faster. They were rerun with a time limit:

Setting	Hop	Outcome	Generation (ms)	Fidelity
dark counts 10^6 /s	0→1	100 samples	1.116	0.950
dark counts 10^6 /s	1→0	100 samples	1.200	0.950
dark counts 10^8 /s	0→1	did not finish in the time limit	–	–
dark counts 10^8 /s	1→0	did not finish in the time limit	–	–
coherence 0.1 ms	0→1	did not finish in the time limit	–	–
coherence 0.1 ms	1→0	did not finish in the time limit	–	–

Dark counts of 10^6 /s (100 samples per hop) agree with the formula within the interval and leave fidelity unchanged, so the rule that dark counts do not matter holds up to 10^6 /s. The other two settings, run with 30 samples and a 7-minute limit, still did not finish: they are too expensive to simulate, so we say nothing about them. A setting that did not finish is part of no conclusion.

Appendix A

Reproducing the study

All commands run from the repository root with the pinned SeQUeNCe engine on `PYTHONPATH` (or `SEQUENCE_ENGINE`).

```
python tools/generation_time_study/build_traces.py            # routes the frozen mappings, checks XY
python tools/generation_time_study/link_sweep.py              # single-link screening
python tools/generation_time_study/anatomy.py --output <data>/anatomy.csv
for e in efficiency delay gap lead_gap tuning memory coherence \
         routes workloads chip scale equivalence distance retry; do
    python tools/generation_time_study/run_matrix.py $e --workers 8; done
python tools/generation_time_study/collect.py                 # data/*.csv
python tools/generation_time_study/choose_tuned.py            # data/tuned_choices.json (tuning seeds
python tools/generation_time_study/run_matrix.py validation   # unseen seeds
python tools/generation_time_study/audit_runs.py              # every case checked against its spec
# then: hypotheses.py, model.py, model_breakeven.py, validate_choices.py
# then: figures.py, make_tables.py
cd docs/latex/generation_time_study/report && tectonic main.tex
```

Appendix B

File map

Path (data files are under `docs/research/generation-time-2026-11-01/`)	Contents
`tools/generation_time_study/`	the code: knobs, runner, experiments, collection, tests (14 pass)
`docs/research/generation-time-2026-11-01/plan.md`	hypotheses, falsification rules, deviations
`data/*_summary.csv`	mean, spread and 95% interval per setting
`data/*_trials.csv.gz`	one row per strategy and seed, 4 112 in all
`data/link_screen.csv`, `data/anatomy.csv`	single-link probes, message-level timeline
`data/hypotheses.json`, `data/model_*.csv`, `data/validation_check.csv`, `data/audit_runs.json`	the analyses
`results/generation_time_2026-11-01/traces/`	the 14 routed traces used
`docs/latex/generation_time_study/report/`	this report, its figures and generated tables

Appendix C

Provenance

Study branch	`abisfa/epr-generation-time-study`
Simulator commit used	`dfd4b839` (main at the start of the study, unchanged under `ace/`)
Engine	SeQUeNCe 0.8.1, commit `cf5283cd`
Python / libraries	3.13; numpy, scipy, pandas, matplotlib
Report built with	tectonic